



Static Range Queries & Count Sort

Presented By : Mazen Othman

- Software Engineer @ Syntech
- Problem Solving Instructor @ Instant
- IEEEExtreme Ambassador @ Egy-Section



Agenda

- Frequency Array
- Count Sort
- Prefix Sum
- 2D Prefix Sum
- Partial Sum



Static Range Queries

Definition . .

- Static Range Queries means that you have some elements, and you are required to perform some queries (Questions) about some range of this elements.
 - For example : you are given an array of size 10 and the query is to get the frequency of specific item in range index 4 to 8 or get the sum of elements from index 3 to index 7
 - For That Type of Problems, it will be helpful to learn some techniques like Frequency Array , Prefix and Suffix Arrays , and Some Additional Tricks that you Might Need
-

Frequency Array

Problem

Given array of size N ($\leq 1e5$) where ($1 \leq 1e5$).

then given Q ($\leq 1e5$) queries

Each Query Given X print The Number Of
Occurrence For Number X

How Can I Solve This Problem ?

Naive Solution :

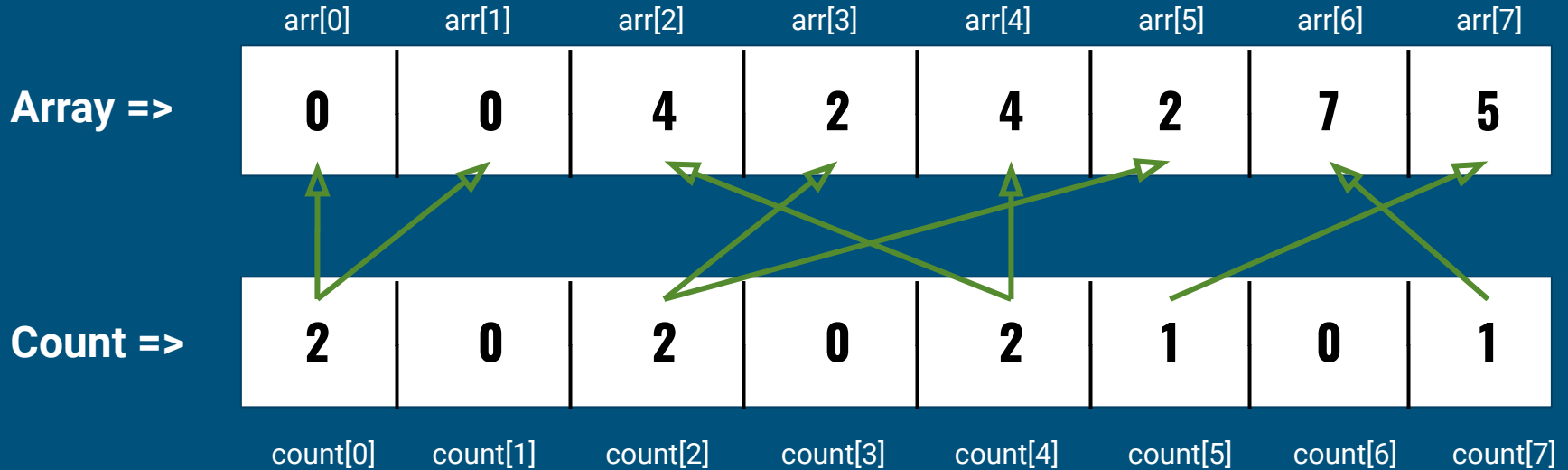
- Every Query, We Can Iterate Over All
Array and Count How many Time[^]
That X Appear.



```
#include <bits/stdc++.h>
using namespace std ;
const int N = 1e5 + 5 ;
int main() {
    int arr[N] ;
    for (int i = 0 ; i < N ; i++){
        cin >> arr[i] ;
    }
    int queries ;
    cin >> queries;
    while (queries--) {
        int x , Counter = 0 ;
        cin >> x;
        for (int i = 0 ; i < N ; i++ ){
            if (arr[i] == x ){
                Counter ++ ;
            }
        }
        cout << Counter << endl;
    }
}
```

Frequency Array

Concept



Frequency Array

Implementation

```
#include <bits/stdc++.h>
using namespace std ;
const int N = 1e5 + 5 ;
int Count[N] = {0} ;
int main() {
    for (int i = 0 ; i < N ; i++){
        int num ;
        cin >> num ;
        Count[num]++ ;
    }
    int queries ;
    cin >> queries;
    while (queries--){
        int x ;
        cin >> x;
        cout << Count[x] <<endl;
    }
}
```

Complexity ?
and Differences



Brainstorming

How Can I Solve This Problem

- When `arr[i] >= 1e9`
 - Memory!



Hint : We Also Have The
Same Number Of Values max
10,000 distinct values



Count Sort

What Is Count Sort

- Counting Sort is a non-comparison-based algorithm that sorts integers.
- It works by counting the occurrences of each distinct value in the input array.

```
void countSort (string &S ) {  
    int count [28] = {0};  
    for (int i = 0 ; i < S.size() ; i++){  
        count[S[i] - 'a']++ ;  
    }  
    char current = 'a' ;  
    int index = 0 ;  
    for (int i = 0 ; i <= 28 ; i++ , current++){  
        int x = count[i] ;  
        for (int j = 0 ; j < x ; j++ ){  
            S[index] = current ;  
            index++ ;  
        }  
    }  
}
```


Cumulative (Prefix) Array

— Problem

Given array of size N ($\leq 1e5$) where ($1 \leq 1e5$) .

then given Q ($\leq 1e5$) queries

Each Query Given L and R print The Sum Of
Numbers From index L to index R

How Can I Solve This Problem

?

Naive Solution :

- Every Query, We Can Iterate Over All Array and Get Sum .

Cumulative (Prefix) Array

What Can We Do To Enhance The Complexity ?

- Let's Create A New Array Such That $\text{Pref}[i] = \text{arr}[0] + \text{arr}[1] + \text{arr}[2] + \dots + \text{arr}[i]$.
- But, $\text{Pref}[i-1] = \text{arr}[0] + \text{arr}[1] + \dots + \text{arr}[i-1]$
- Then, $\text{Pref}[i] = \text{Pref}[i-1] + \text{arr}[i]$

	arr[0]	arr[1]	arr[2]	arr[3]	arr[4]	arr[5]	arr[6]	arr[7]
Array =>	1	2	3	8	4	3	8	7
Pref =>	1	3	6	14	18	21	29	36
	Pref[0]	Pref[1]	Pref[2]	Pref[3]	Pref[4]	Pref[5]	Pref[6]	Pref[7]

Cumulative (Prefix) Array

Implementation

```
int arr[10] = {1,2,3,4,5,6,7,8,9,10};
int pref[10] = {0};
pref[0] = arr[0] ;
for (int i = 1 ; i < 10 ; i++){
    pref[i] = pref[i-1] + arr[i] ;
}
for(int i = 0 ; i < 10 ; i++){
    cout << pref[i] << " " ;
}
```

Complexity ?



Cumulative (Prefix) Array

Problem

Given String with lowercase characters of size N ($\leq 1e5$)

then given Q ($\leq 1e6$) queries

Each Query Given L and R and X

print The Number of times that X Appear in Range $[L : R]$

How Can I Solve This Problem

?

Naive Solution :

- Every Query, We Can Iterate Over Range $[L : R]$ and Count The Times That X Appear.

- Sum From $(4, 6)$?

→ it's equal to sum from $(0, 6)$ - sum from $(0, 4)$.
→ So, $\text{Sum}(L, R) = \text{Sum}(0, R) - \text{Sum}(0, L)$.

Problem Solution

Implementation

```
string S ;
cin >> S ;
int n = S.size() , Q , L , R ;
char ch ;
cin >> Q ;
int Pref [28][n] ;
for (int i = 0 ; i < n ; i++){
    for (int j = 0 ; j < n ; j++){
        Pref[i][j] = 0 ;
    }
}
for (int i = 0 ; i < n ; i++){
    ch = S[i] - 'a' ;
    Pref[ch][i] = 1 ;
}
for (int i = 0 ; i < n ; i++){
    for (int j = 1 ; j < n ; j++){
        Pref[i][j] += Pref[i][ j - 1 ] ;
        // Pref[i][j] = Pref[i][j] + Pref[i][ j - 1 ] ;
    }
}
while (Q--){
    cin >> ch >> L >> R ;
    cout << Pref[ch - 'a'][R] - Pref[ch - 'a'][L-1] << endl ;
}
```

Complexity ?



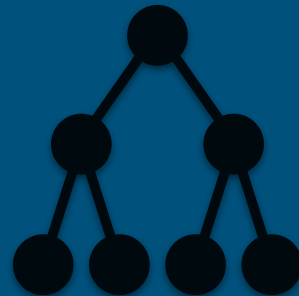


Any Questions

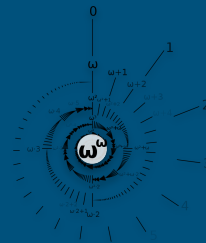




CODEFORCES



Thank
You



LeetCode

